# WPRDC Property Sales Pipeline

A daily pipeline that downloads Allegheny County property sale records from the Western Pennsylvania Regional Data Center (WPRDC), lands them unchanged in Postgres, cleans them with dbt, and builds one table that answers a question about home prices.

Built step by step, by hand, between 2026-10-26 and 2026-10-27.

---

## The question

**How has the median home sale price changed in each municipality between 2019 and 2025?**

**Answer:** among municipalities with at least 30 valid sales in both years, **Penn Hills** rose the most:

| | valid sales | median price |
|---|---|---|
| Penn Hills 2019 | 135 | $89,500 |
| Penn Hills 2025 | 326 | $184,575 |
| **Change** | | **+$95,075 (+106.23%)** |

Answer table: `analytics.mart_muni_year_prices` (one row per municipality per year).

---

## Stack

| Piece | What it does | Where |
|---|---|---|
| Postgres 16 (Docker) | the warehouse: `raw` and `analytics` schemas | `docker-compose.yml`, service `postgres` |
| Python (`requests`, `psycopg2`) | download the CSV and load it into `raw.sales` | `ingest.py` |
| dbt 1.12.6 + dbt-postgres 1.11.0 | clean, model, test | `dbt/` |
| Airflow 3.3.2 (Docker) | runs ingest, then dbt, every day at 09:00 UTC | `Dockerfile`, `dags/wprdc_sales.py` |

---

## How it works

```
WPRDC CKAN API
   │  1. resource_show → gives the download URL
   │  2. download the full CSV (~506k rows)
   ▼
data/raw/sales.csv              ← the file exactly as received
   │  TRUNCATE + COPY, one transaction, with checks
```

```
   ▼
raw.sales                        ← every column TEXT, unchanged
   │  dbt
   ▼
analytics.stg_sales              ← typed, names trimmed, all rows kept
   │  dbt
   ▼
analytics.mart_muni_year_prices ← VALID SALE only, median per town per year
```

Airflow DAG `wprdc_sales`: `ingest >> dbt_build`, daily at `0 9 * * *`.

---

## How to run

```
docker compose up -d --build           # starts Postgres and Airflow
open http://localhost:8080             # Airflow UI → turn on wprdc_sales →
Trigger
```

Query the result in DBeaver (PostgreSQL, `localhost:5433`, database `warehouse`; user and password are in `docker-compose.yml`):

```
SELECT * FROM analytics.mart_muni_year_prices
WHERE municipality = 'Penn Hills'
ORDER BY sale_year;
```

Running steps by hand, without Airflow:

```
# on the Mac (uses localhost:5433)
source .venv/bin/activate
python ingest.py
cd dbt && dbt build --profiles-dir .

# inside the Airflow container (uses postgres:5432)
docker compose exec airflow python /opt/airflow/ingest.py
docker compose exec airflow /home/airflow/dbt-venv/bin/dbt build --project-
dir /opt/airflow/dbt --profiles-dir /opt/airflow/dbt
```

---

## Files

| File | What it does |
|---|---|
| `docker-compose.yml` | two services, `postgres` and `airflow`, on one private network. Postgres is on `5433` on the Mac (`5433:5432`), Airflow UI on `8080`. |

| File | What it does |
|---|---|
| | Mounts `dags/`, `ingest.py`, `dbt/`, `data/` into the Airflow container |
| `Dockerfile` | `FROM apache/airflow:3.3.2-python3.12`, plus dbt in its **own venv** at `/home/airflow/dbt-venv` (Airflow and dbt need different versions of shared libraries) |
| `requirements-dbt.txt` | the two pinned dbt packages for the image |
| `requirements.txt` | `pip freeze` of the Mac venv (local use only, not used by the image) |
| `ingest.py` | `get_download_url()` → `download(url)` → `load(path)` |
| `dags/wprdc_sales.py` | the DAG: 2 `BashOperator` tasks, retries |
| `dbt/dbt_project.yml` | project `wprdc`, models built as tables |
| `dbt/profiles.yml` | connection; host/port from `DB_HOST` / `DB_PORT`, falling back to `localhost` / `5433` |
| `dbt/models/sources.yml` | declares `raw.sales` as a source |
| `dbt/models/staging/stg_sales.sql` | types and trims |
| `dbt/models/marts/mart_muni_year_prices.sql` | the answer table |
| `dbt/models/schema.yml` | 3 `not_null` tests on the mart |
| `dbt/tests/*.sql` | 2 custom tests |

## `ingest.py` in detail

- **`get_download_url()`**: calls `resource_show?id=5bbe6c55-bce6-4edb-9d04-68edeb6bf7b1` (60 s timeout). Fails on a bad HTTP status or `success != true`. Returns `result.url`.
- **`download(url)`**: streams the file in 1 MB chunks (never holds 110 MB in memory) to `sales.csv.tmp`, then `os.replace` to `sales.csv`. If anything fails, the `.tmp` is deleted. The rename is atomic, so `sales.csv` is never half-written.
- **`load(path)`**:
  1. Reads the header and compares it with the 26 `EXPECTED_COLUMNS`. Any difference stops the run.
  2. Opens the file in text mode (`newline=None`), so the source's Windows `\r\n` line endings become `\n` before COPY.
  3. In **one transaction**: creates `raw.sales` (all `text`) if missing → counts the old rows → `TRUNCATE` → `COPY` → counts the new rows.
  4. Raises (and so rolls back everything, including the TRUNCATE) if the new count is 0, or below 90% of the old count.

Environment: Mac vs container

| dbt / ingest runs on | host | port | set by |
|---|---|---|---|
| the Mac | `localhost` | `5433` | the defaults in code |
| the Airflow container | `postgres` (service name) | `5432` | `DB_HOST`, `DB_PORT` in `docker-compose.yml` |

The `ports:` mapping is only for the Mac. Containers reach each other by service name on the internal port.

---

## Design decisions (and what I rejected)

**1. The raw layer stores every column as text, exactly as received.** Rejected: typing columns while loading (e.g. `price numeric`). Why: one bad value would fail the whole load. In text, everything lands, and the conversion happens in dbt where it can be checked and counted.

**2. Full reload with TRUNCATE + COPY in one transaction.** Rejected: appending rows (running twice would double the data) and running TRUNCATE and COPY as separate steps (a crash in between leaves an empty table). Why: the run is **idempotent** (any number of runs gives the same result), and readers see the old data until the new data commits. COPY instead of row-by-row INSERT: one bulk stream instead of 506,000 round trips.

**3. Don't trust a download just because it returned HTTP 200: check it.** Rejected: accepting whatever the server sends. Why: the source returned a 502 error page, and later a complete-looking file missing 21% of its rows. The header check and the row-count check (no drop over 10% vs. the last load) caught it, and the old data stayed in place.

Smaller decisions:

- **Median, not average.** One $5M deal wrecks an average; the median shows the typical sale.
- **Keep small town-years in the mart, with a `sales` column**, and filter `sales >= 30` when comparing. Rejected: `HAVING count(*) >= 30` in the mart, which hides data.
- **dbt in its own venv** inside the Airflow image, to avoid library version conflicts.
- **Mount code into the container** instead of `COPY`, so edits show up without rebuilding.
- **ingest retries 3× (5 min apart); dbt_build retries 0×.** Network errors are temporary; a failed dbt test is the data and will fail again.
- **09:00 UTC schedule.** WPRDC republishes around 07:00 UTC (`last_modified`).

---

## Data quality checks

| Check | Where | Fails when |
|---|---|---|
| Header = the 26 expected columns | `ingest.py` | WPRDC renames, adds or removes a column |
| Rows > 0 | `ingest.py` | empty file |

| Check | Where | Fails when |
|---|---|---|
| Rows ≥ 90% of previous load | `ingest.py` | truncated download, or the source lost data |
| `municipality_is_trimmed` **(dataset-specific)** | `dbt/tests/` | a municipality name still has leading/trailing spaces (the source pads them, e.g. `"Penn Hills "`) |
| `mart_one_row_per_town_year` | `dbt/tests/` | duplicate `(municipality, sale_year)` rows in the mart |
| `not_null` on municipality, sale_year, median_price | `schema.yml` | missing values in the answer table |

`dbt build` runs each model's tests before building the next model, so a failing staging test **skips the mart**. Verified: removing `TRIM()` made `municipality_is_trimmed` fail and the mart was skipped.

---

## Log, by date

### 2026-10-26: Postgres, exploring the source, ingest

| Step | What happened |
|---|---|
| Postgres in Docker | `postgres:16` with a named volume, port `5433:5432`. `SELECT version()` works from DBeaver |
| Explored the API | `resource_show` returns metadata; `result.url` is the CSV dump |
| **Issue:** saved metadata instead of data | first `curl` used the API URL, so `sample.csv` got JSON. Lesson: call 1 gives the URL, call 2 fetches the file |
| **Issue:** 502 Bad Gateway | WPRDC's dump endpoint was down. `curl` saved the HTML error page as if it were CSV. Fix: `curl -f`, `--retry`; in Python `raise_for_status()` |
| **Issue:** `head -20` still pulled the whole file | the server prepares the response before sending. Used the full file instead |
| `ingest.py` download | 506,426 lines = 1 header + **506,425 rows** |
| Review fixes | added `timeout=60` to the API call (could hang forever); close the streamed response |
| **Issue:** `BadCopyFileFormat: unquoted newline found in data` | the file uses Windows `\r\n` line endings (`od -c` showed `\r\n`). Fix: open the file in text mode so Python converts them to `\n` before COPY |
| Load | `raw.sales` = 506,425 rows. Ran twice: still 506,425 (idempotent) |

## 2026-10-27: checks, dbt, Docker image, Airflow

| Step | What happened |
|---|---|
| Header + row-count checks | test with a 1,000-row file: `RuntimeError: Loaded row count dropped from 506425 to 999`, and `raw.sales` stayed at 506,425 |
| Explored the data in DBeaver | see "Observations" below |
| dbt setup | `dbt debug` passes with `localhost:5433` |
| `stg_sales`, `mart_muni_year_prices` | `dbt run` order is worked out from `ref()`: staging first, then the mart |
| Tests | `dbt build`: PASS=7 (2 models + 5 tests). Removing `TRIM` → trim test fails → mart skipped |
| Dockerfile | builds; image is 3.6 GB (mostly the Airflow base image) |
| **Issue:** `docker run wprdc-airflow dbt --version` printed Airflow help | the image's entrypoint passes arguments to `airflow`. Fix: `bash -c "..."` or `--entrypoint` |
| **Issue:** test line left in code | `load("data/raw/short.csv")` was still in `__main__`, so every run failed the row-count check. Fixed back to `load(path)` |
| **Issue:** `connection to server at "localhost", port 5433 failed` inside the container | `DB_HOST` / `DB_PORT` were not reaching the container. Fixed; the container then connected to `postgres:5432` |
| **Issue:** source lost 21% of its rows | see below |
| DAG | `wprdc_sales` runs; `ingest` fails (bad source) → up_for_retry → retries; `dbt_build` is upstream_failed and never runs. Old data stays in place |

**The 2026-10-27 source problem**

Every download returned **397,885 rows instead of 506,425**. The row-count check refused all of them.

Investigation:

- The API itself reported `"total": 397885`, so this was **not** a truncated download.
- The curl download was complete (82 MB, 8 s) and ended on a complete row.
- Rows per sale year, old load vs. new file:

| Year | Old (in raw.sales) | New file |
|---|---|---|
| 2012–2021 | 33,651 … 40,228 | nearly identical (each year −10 to −90) |
| 2022 | 34,825 | 33,259 |
| 2023 | 30,859 | **4,653** |

| Year | Old (in raw.sales) | New file |
|---|---|---|
| 2024 | 31,111 | **4,740** |
| 2025 | 31,551 | **509** |
| 2026 | 23,063 | **3** |

Conclusion: the recent years almost disappeared while older years were unchanged. That looks like a broken or half-finished publish at WPRDC, not a real change. **Decision: do not load it.** Loading it would have made the 2019 → 2025 comparison impossible, with every dbt model still "succeeding". The previous good 506,425 rows stay in `raw.sales`.

---

## Observations about the data

1. **Only 19.78% of rows are `VALID SALE`.** The file records every change of owner: gifts within families, sheriff and bank sales, quit claims, multi-parcel deals. Only valid sales show the market price.
2. **Prices of $0 and $1 are common.** These are nominal prices on gifts and transfers, not errors, and not sales.
3. **Municipality names are padded with spaces.** `"Penn Hills"` has length 12, not 10. Without `trim()`, one town could split into two groups.
4. **Multi-parcel deeds repeat the full price on every parcel.** One deed (deed book + page) for 10 parcels at $5M becomes 10 rows of $5M. Summing would overcount 10×. These deeds are labelled `MULTI-PARCEL SALE` / `LOVE AND AFFECTION`, not `VALID SALE`, so the valid-sale filter removes them.
5. **Placeholder deed numbers.** Deed book `0` / `00` / blank means "unknown"; thousands of unrelated transfers share it, with different prices and descriptions. Book + page is not a key for those rows.
6. **Tiny town-years give meaningless medians.** One town-year had a median of **$500** from a **single** sale. Comparisons use town-years with at least 30 sales.
7. **The file uses Windows `\r\n` line endings**, which broke the first COPY.
8. **The description says "2013-present", but the data starts in 2012.**
9. **The description says "updated on the first business day of each month", but `last_modified` changes daily** (e.g. 2026-10-26T07:08 UTC).
10. **The source is unreliable:** 502 errors on 2026-10-26, and a 21% row loss on 2026-10-27, both returned without a useful error.

---

## What changes at 100× scale / with daily refreshes

- **Full download every day stops making sense.** 50 million rows is about 11 GB per day. Fetch only what changed (CKAN `datastore_search` with a filter on a date or `_id`), or check `last_modified` first and skip the run if nothing changed.
- **TRUNCATE + COPY of the whole table gets slow** and holds a lock for the whole load. Load into a new table and swap it in, or partition by year and replace only changed partitions.
- **dbt full rebuilds get slow.** Make `stg_sales` incremental, and only rebuild mart years that changed.

- **The 10% row-drop rule is too blunt.** Compare per year (as in the investigation above), so a loss in recent years is caught even if the total barely moves.
- **One file on disk is not enough history.** Keep dated or hashed raw files in object storage (S3 / GCS) to replay or compare days.
- **Airflow `standalone` is for local use only.** Use a real scheduler + workers + a Postgres metadata DB, or a managed Airflow.

---

## Known issues and what I didn't do

| # | Item | Status |
|---|---|---|
| 1 | **`.gitignore` is missing `data/`, `__pycache__/` and `dbt/.user.yml`.** `data/raw/sales.csv` is 86 MB and must never be committed (GitHub rejects files over 100 MB) | **fix before `git init`** |
| 2 | **A rejected download still overwrites `data/raw/sales.csv` on disk.** The file there now is the bad 397,885-row one; the good data exists only in `raw.sales` | to do: name files by date or hash, keep the last N |
| 3 | No `--force` flag to accept a real, checked drop in rows | to do |
| 4 | No alert when the DAG fails; you only notice in the UI | not done |
| 5 | No check that the mart's 2025 numbers look complete (e.g. per-year row counts) | to do |
| 6 | Credentials are hard-coded in `ingest.py`, `profiles.yml`, `docker-compose.yml` (local only) | to do: read from environment |
| 7 | `ingest.py` has no retry of its own; it relies on Airflow retries | fine for now |
| 8 | Leftover test files: `data/raw/short.csv`, `data/sample.csv` | delete |
| 9 | No tests for `ingest.py` itself (only manual tests: short file, run twice) | to do |
| 10 | Not yet a git repository | `git init` after #1 |